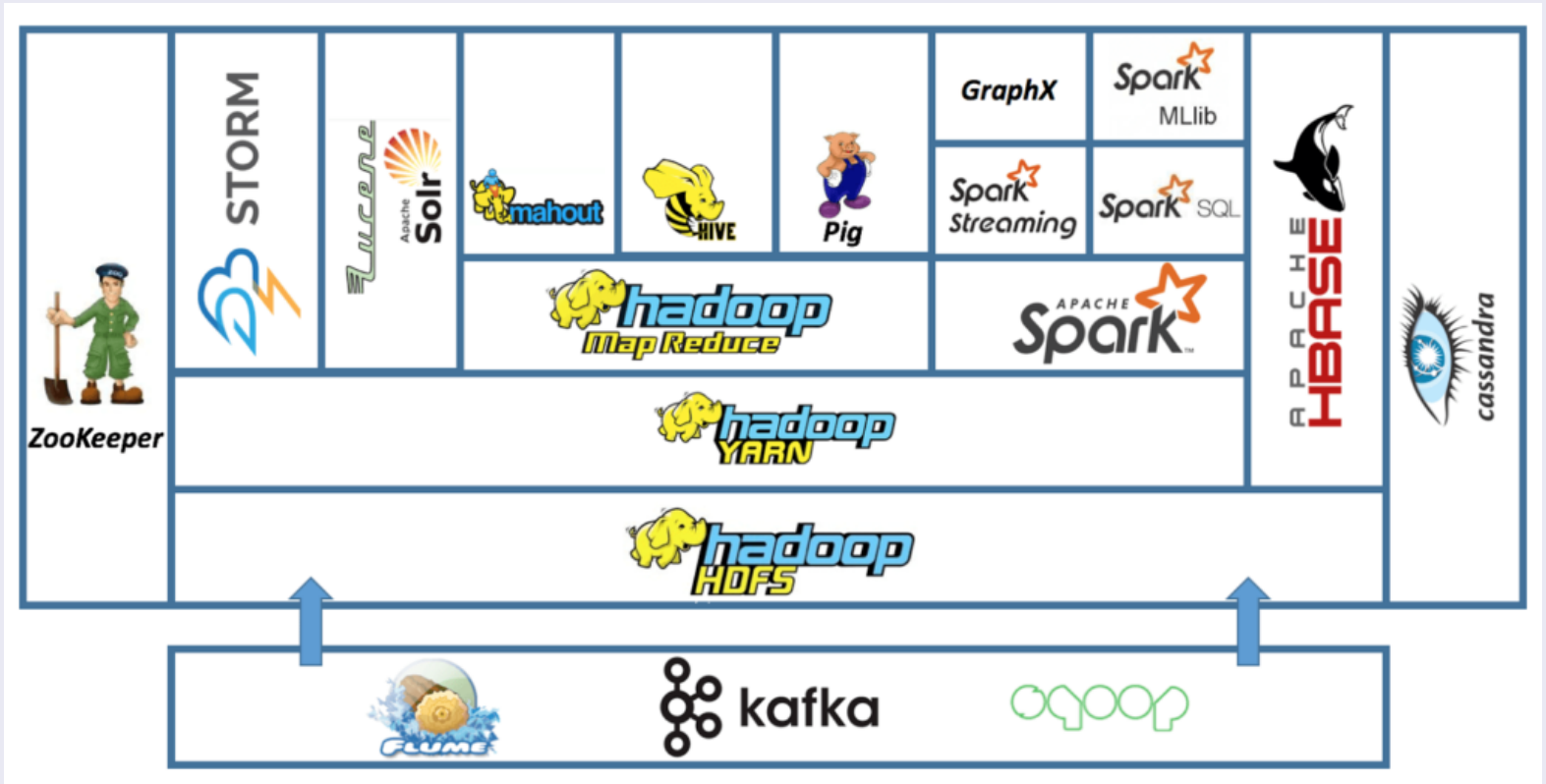
Wiodące projekty Big Data

Dominik Ryżko

Wydział Elektroniki i Technik Informacyjnych, Politechnika Warszawska

Ekosystem Hadoop

Ekosystem



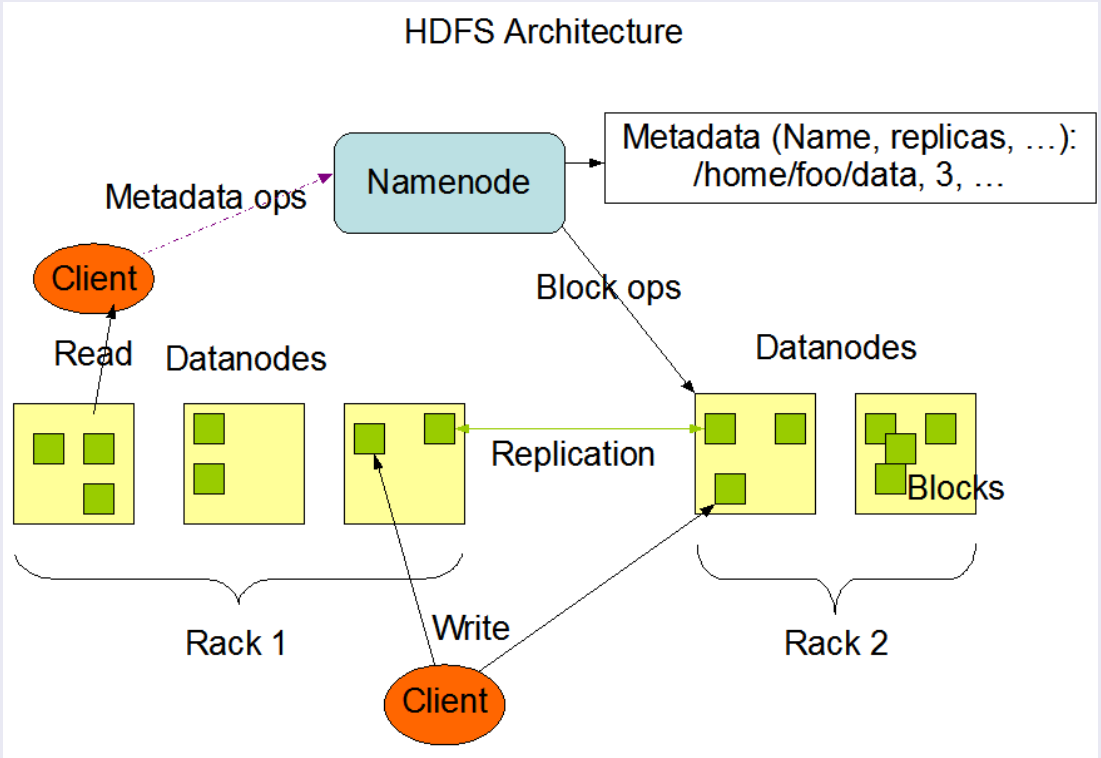
Hadoop

Założenia:

- Odporność na awarie sprzętu
- Optymalizacja przepustowości kosztem responsywności (dane strumieniowe)
- Wielkie zbiory danych (duże pliki)
- Prosty model spójności
- Przenoszenie obliczeń jest tańsze niż migracja danych
- Przenośność

HDFS

HDFS


HDFS Architecture
Namenode
Metadata (Name, replicas, …):
/home/foo/data, 3, …
Metadata ops
Client
Block ops
Read
Datanodes
Datanodes
Replication
Blocks
Rack 1
Write
Rack 2
Client

HDFS

NameNode

- Zarządzanie systemem plików (struktura drzewiasta)
- mapowanie plików na bloki
- kontrola dostępu

DataNode

- Zarządzanie przestrzenią dyskową
- Przechowywanie bloków danych

HDFS

Replikacja danych

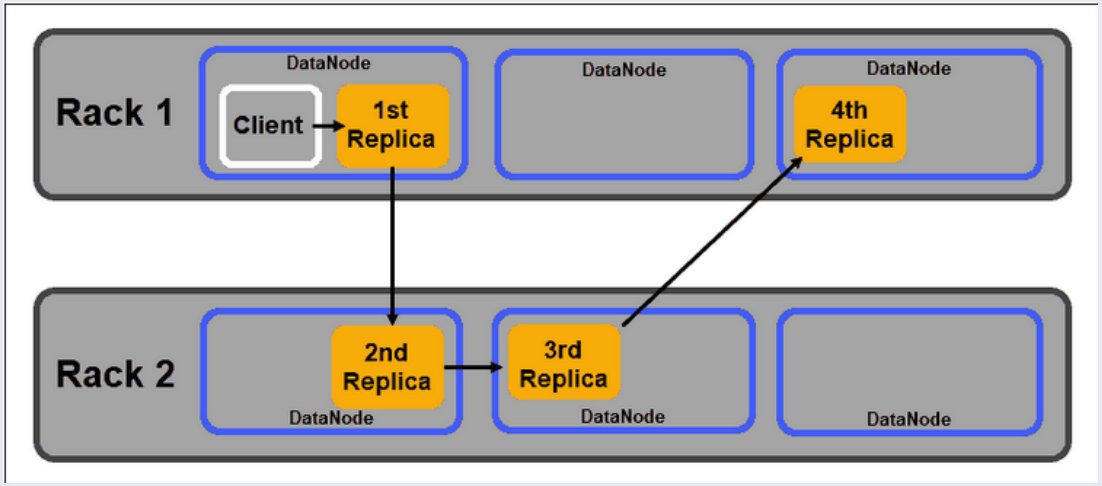
- Współczynnik replikacji można ustalić na poziomie pojedynczego pliku
- Proces zapisu bloków określa się jako *pipelined* - pierwszy węzeł otrzymuje i zapisuje dane, po czym przekazuje do kolejnego węzła, który ma stworzyć następną replikę etc.

Niezawodność vs wydajność

Optymalny sposób rozdziału bloków na węzły jest zadaniem nietrywialnym i zależy od konfiguracji całego klastra, w tym jego warstwy fizycznej. Maksymalne rozproszenie bloków pomiędzy urządzeniami (nawet szafami w serwerowni) zwiększa niezawodność, jednak obniża czas zapisu.

HDFS

Rack Aware Placement Policy



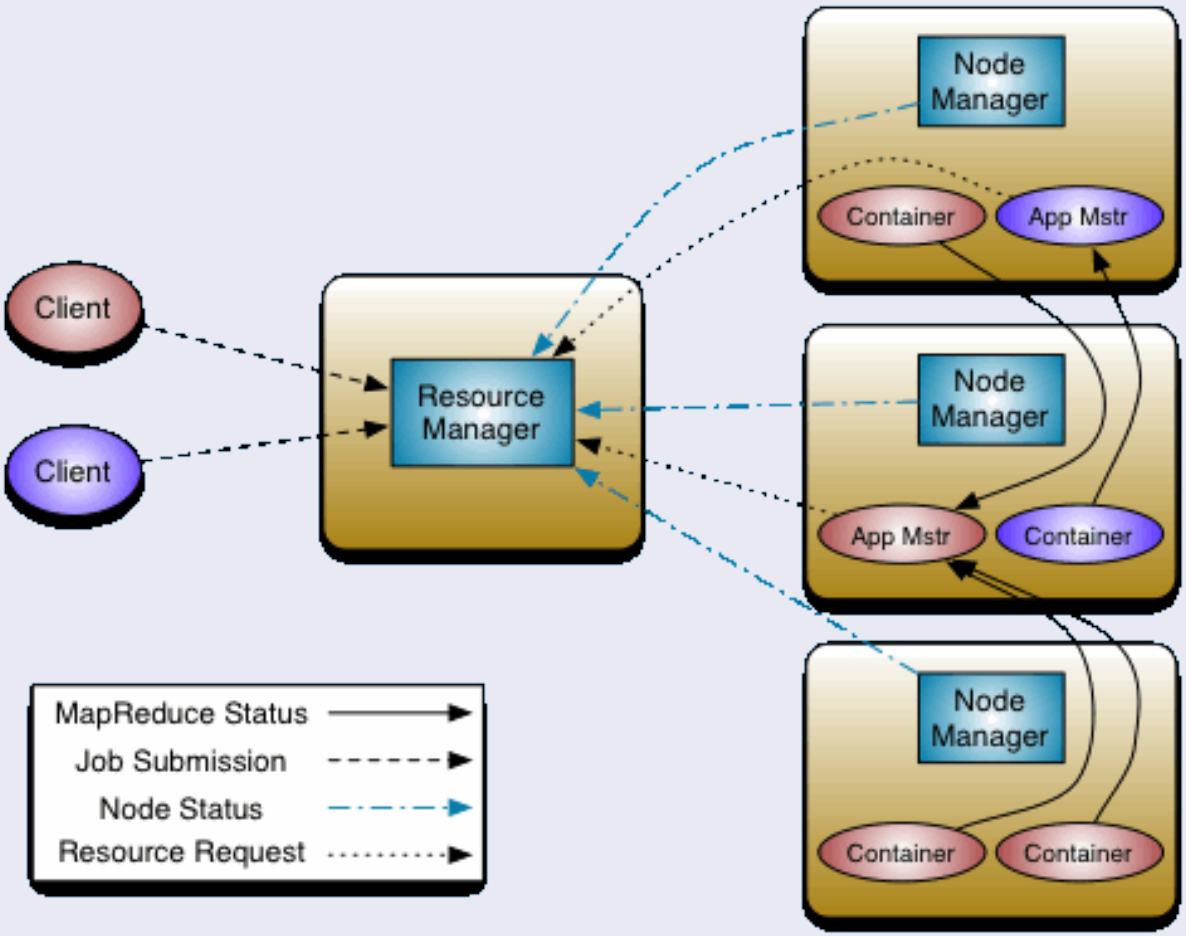
Polityka replik:

- Pierwsza replika bloku umieszczana jest na tym samym węźle co klient
- Druga replika wysyłana jest do losowego DataNode ona innym racku
- Trzecia replika umiszczana jest na innym DataNode w tym samym racku co 2-ga
- Kolejne repliki rozmieszczane są losowo

Yarn - Yet Another Resource Negotiator

- Odpowiada za zarządzanie zasobami w Hadoopie
- Komponenty:
  - Resource Manager - jeden na system - jest sercem całego systemu
  - Application Master - jeden na aplikację - monitoruje pracę danej aplikacji
    - Scheduler - Układa plan wykonania zadań, nie prowadzi monitorowania ich pracy
    - Applications Manager - Przy starcie aplikacji, zapewnia uruchomienie jej Application Mastera i monitoruje dalej jego pracę (ew. restarty)
  - Node Manager - jeden na maszynę - zarządza kontenerami, monitoruje zasoby (CPU, RAM, HDD, sieć) i raportujes status do Resource Managera
- Pozwala się skalować przez łączenie wielu instancji

Yarn


Client
Client
Resource Manager
Node Manager
Container
App Mstr
Node Manager
App Mstr
Container
Node Manager
Container
Container
MapReduce Status
Job Submission
Node Status
Resource Request

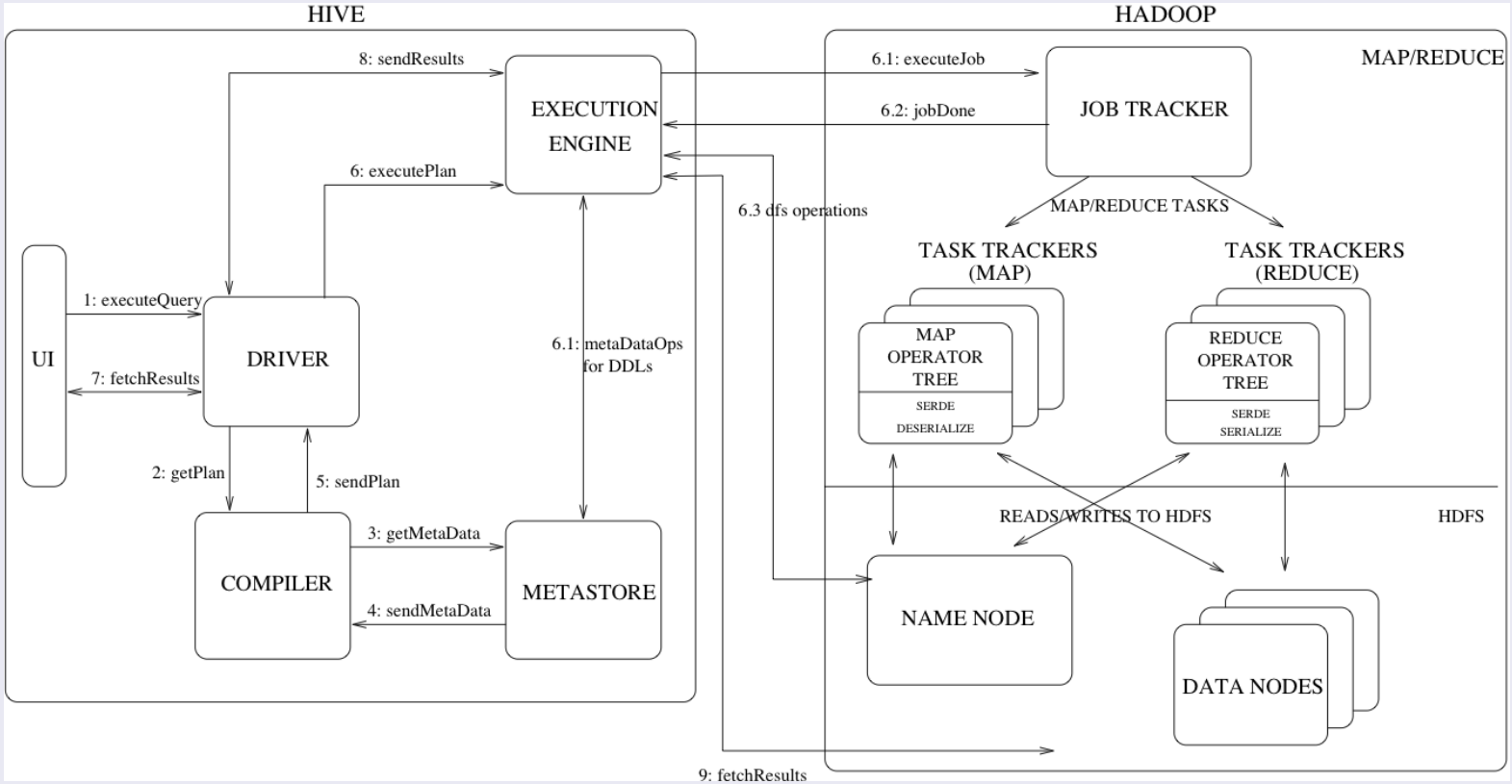
Hive

Dostarcza możliwość odpytywania i analizowania dużych zbiorów danych przechowywanych w systemie Hadoop.

Elementy:

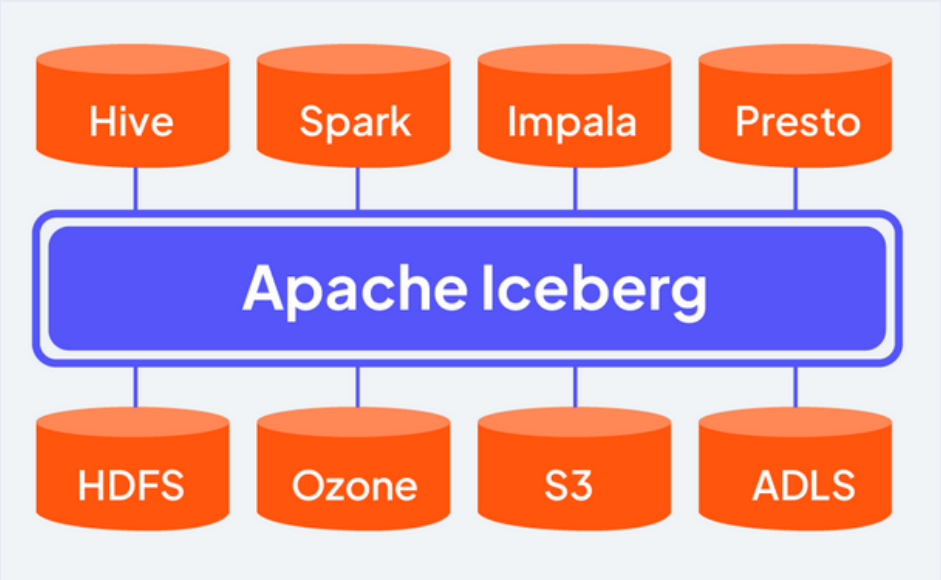
- Metastore - Przechowuje metadane - m.in. pliki, tablice, partycje, kolumny, ich typy etc.
- Sterownik - Komponent odbierający zapytania i zarządzający ich cyklem życia
- Kompilator - Parsuje zapytania i generuje plan ich wykonania
- Silnik - Wykonuje plan ułożony przez kompilator

Hive


HIVE
HADOOP
MAP/REDUCE
8: sendResults
6.1: executeJob
6.2: jobDone
EXECUTION ENGINE
JOB TRACKER
6: executePlan
6.3 dfs operations
MAP/REDUCE TASKS
TASK TRACKERS (MAP)
TASK TRACKERS (REDUCE)
UI
1: executeQuery
7: fetchResults
DRIVER
6.1: metaDataOps for DDLs
MAP OPERATOR TREE
SERDE DESERIALIZE
REDUCE OPERATOR TREE
SERDE SERIALIZE
2: getPlan
5: sendPlan
READS/WRITES TO HDFS
HDFS
3: getMetaData
COMPILER
METASTORE
4: sendMetaData
NAME NODE
DATA NODES
9: fetchResults

Iceberg

Nowoczesny, ACID-owy system zarządzania tabelami w data lake.



Iceberg

Rozwiązane problemy:

- Pełna transakcyjność ACID
- Atomowe operacje na danych
- "Time travel"
- elastyczne zarządzanie schematem
- wydajne partycjonowanie

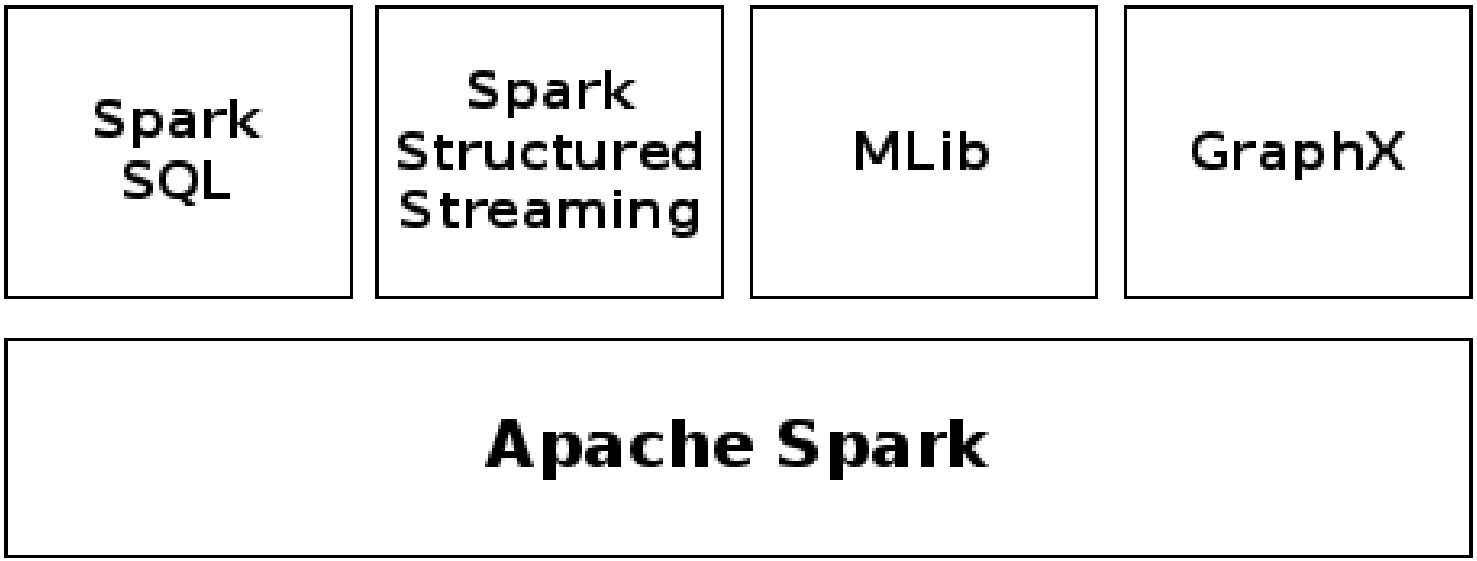
Spark

Założenia:

- In-Memory - zorientowany na przetwarzanie w pamięci
- Szybkość - znaczące przyspieszenie w stosunku do modelu MapReduce
- Cachowanie - Optymalizacja wykorzystania danych załadowanych do pamięci
- Otwartość na systemy zarządzania - Spark, Yarn, Mesos
- Otwartość na języki programowania - API dostępne dla Java, Scala, Python, R

Spark

Architektura



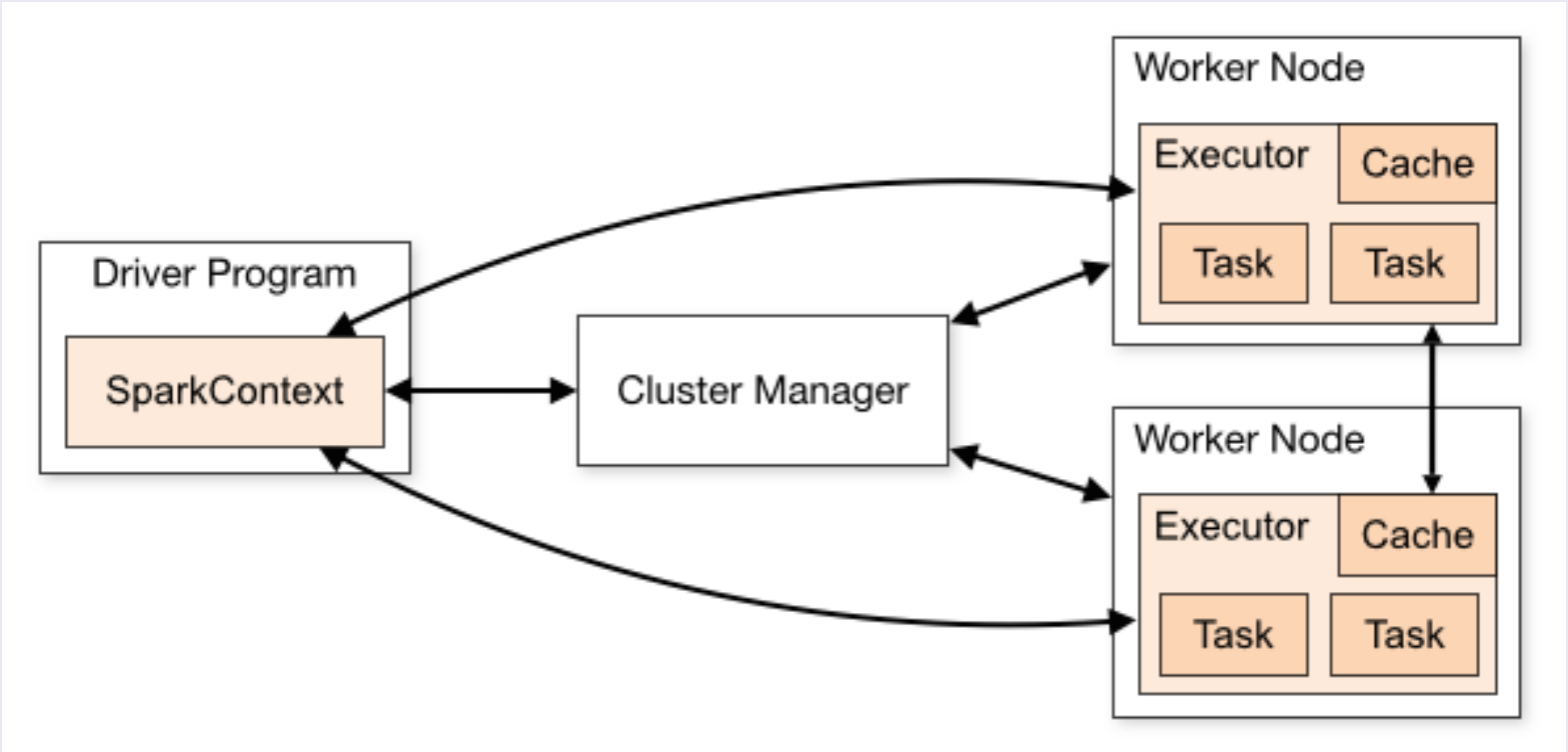
Spark

Komponenty:

- SparkContext - "serce" aplikacji w Spark
- Spark Driver - zawiera funkcję main(), tworzy RDDs oraz SparkContext
- Cluster Manager - Uruchamia executory
- Executory - uruchamiane jednorazowo przy starcie aplickaji i działające do jej zakończenia

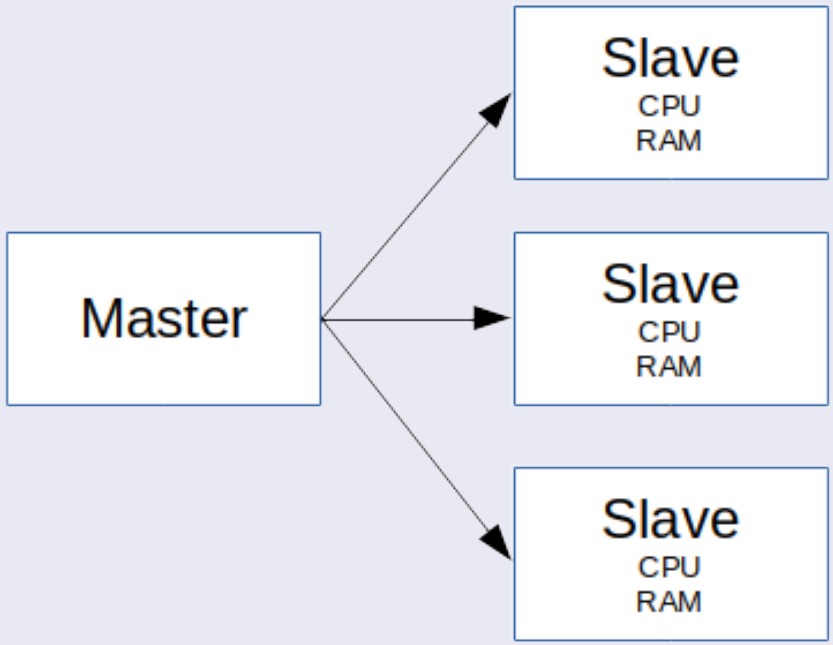
Spark

Architektura



Spark

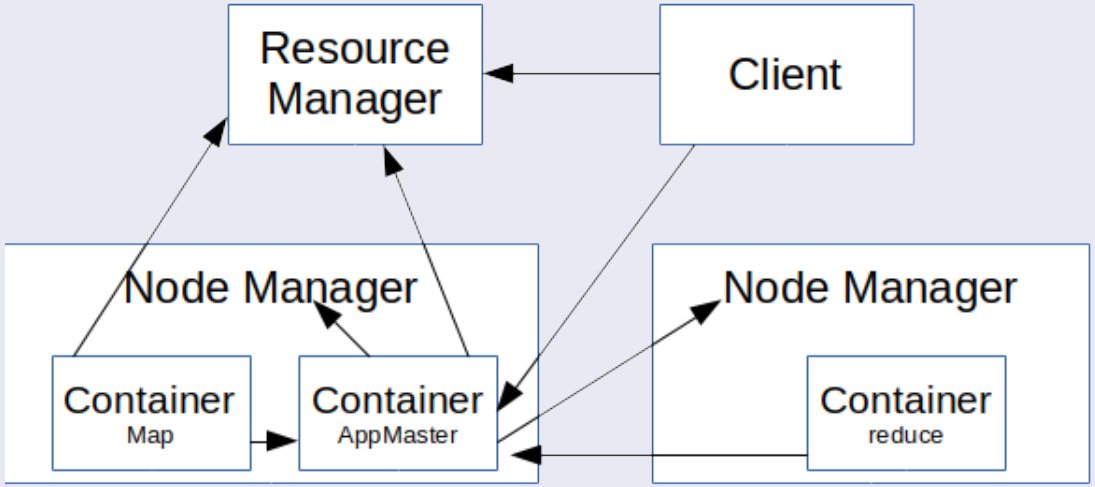
Architektura *standalone*



- **Standalone Master**, **Standaone Worker**
- Jeden egzekutor na węzeł

Spark

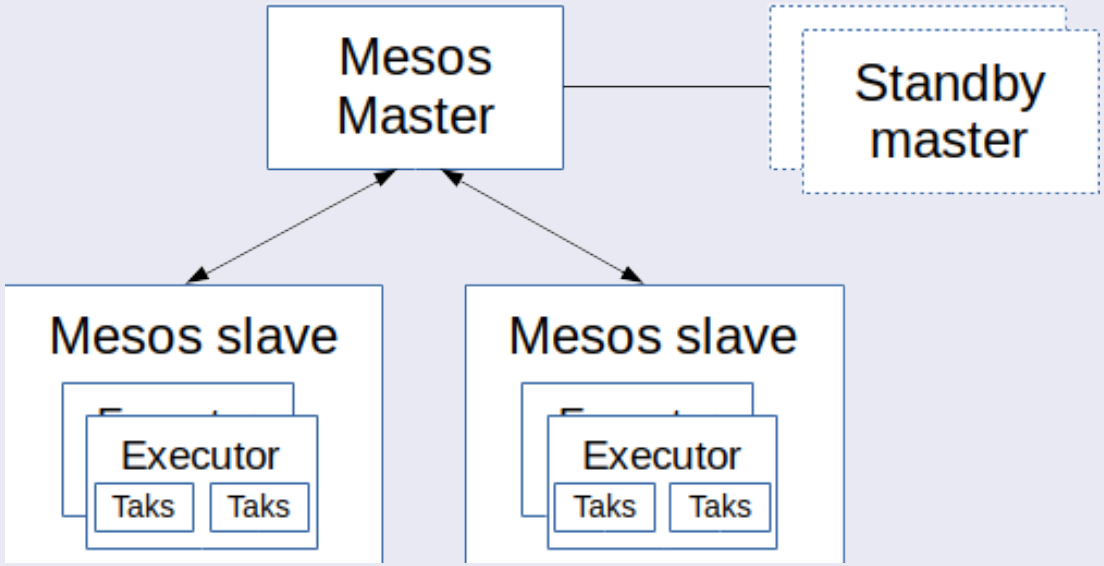
Architektura z Yarn



- Yarn przejmuje zarządzanie zasobami
- W każdym węźle mamy dedykowanego managera
- Zadania uruchamiane w osobnych kontenerach

Spark

Architektura Mesos



- Mesos Master - zapenia zarządzanie kalstrem w bezawaryny sposób
- Mesos Slave - udostępnia zasoby po otrzymaniu zapotrzebowania z Mastera

Spark

Resilient Distributed Dataset(RDD)

- Resilient - można odtworzyć dane w razie awarii
- Distributed - rozproszenie danych w klastrze
- Dataset - kolekcje partycjonowanych danych

RDDs są trwałe (immutable). Można wykonywać na nich transformacje i tworzyć nowe RDD.

Data Frame

- Trwałe kolekcje
- Dane zorganizowane w nazwane kolumny
- Łatwe do operacji w SQL, R

Dataset

- Rozszerzenie DataFrame
- Dostarcza obiektowego API

Storm

Podstawowym pojęciem w Stormie są **topologie** - schematy obliczeniowe modelowane jako DAG (skierowane gafy acykliczne). Topologie przetwarzają **strumienie** - sekwncje krotek (listy nazwanch wartości). Strumienie są nieograniczone, nie zakłada się możliwości ich całego pobrania przed rozpoczęciem obliczeń.

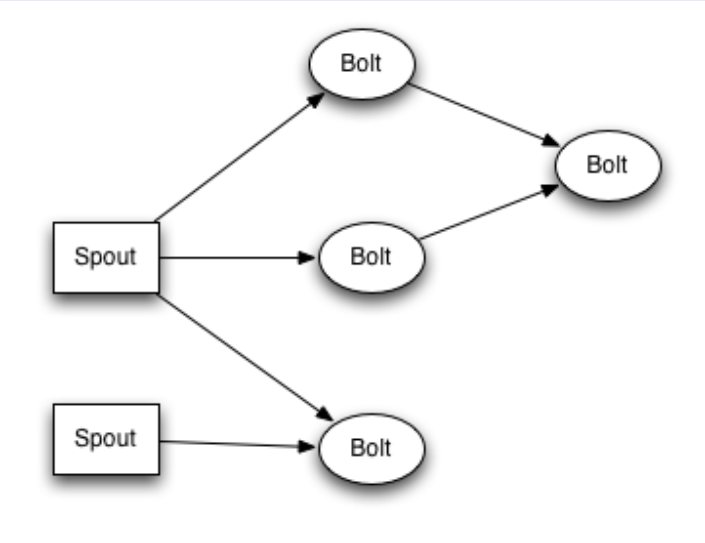
Spouts

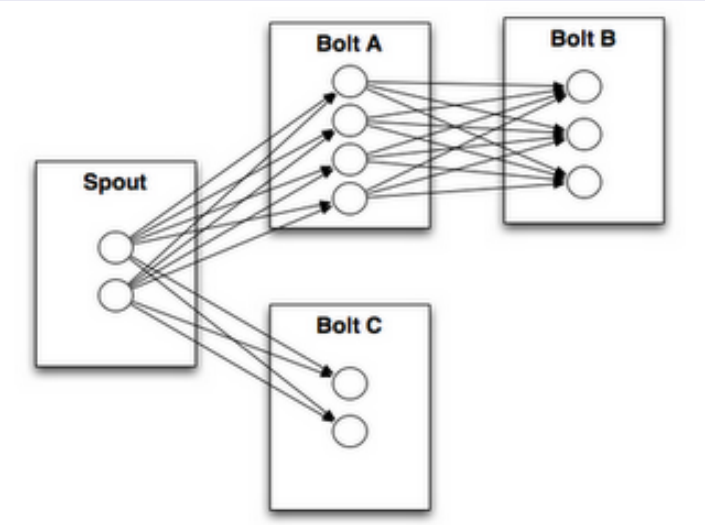
Elementy wejściowe topologii. Biorą wejście (np. API) i generują strumień.

Bolts

Implementują transformacje na strumieniach np. filtrowanie, złączenie etc. Działają asynchronicznie i mogą być skalowane niezależnie.

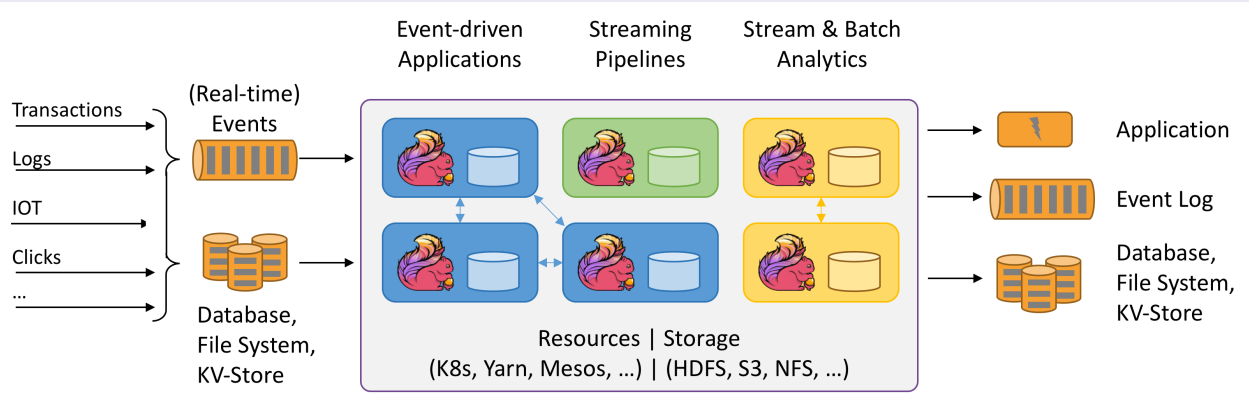
Storm





Flink

- Rozróżnia strumienie ograniczone (bounded) i nieograniczone (unbounded). Te pierwsze można pobrać w całości, przy czym kolejność przetwarzania nie ma znaczenia, co pozwala modelować procesy batchowe.
- Posiada bibliotekę wspierającą wykrywanie wzorców w danych (CEP)
- Przetwarza dane rekord po rekordzie, ale ma mechanizm okien czasowych



Spark vs Storm vs Flink

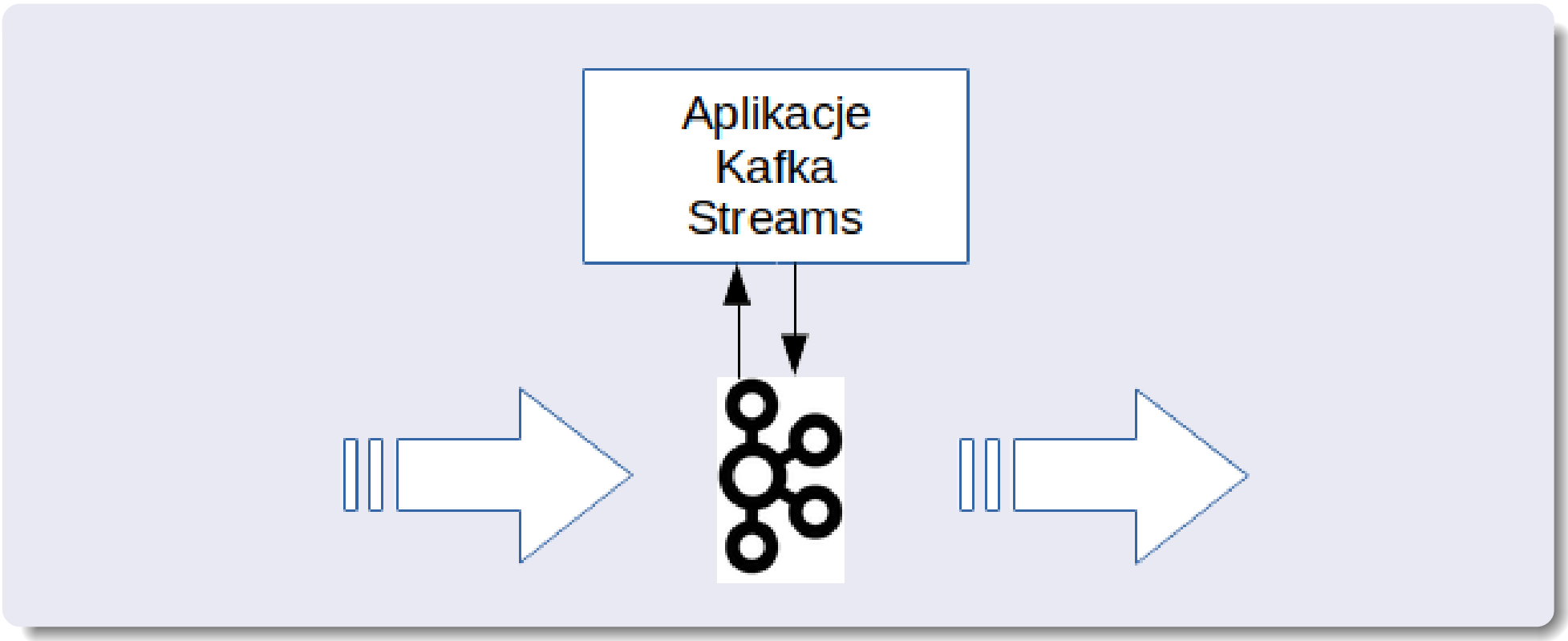
	Spark	Flink	Storm
Throughput	High	High	High
Deduplication	Yes	Yes	Yes
Comp. Model	Micro batch	Stream	Stream
Window	Time based	Record or custom	Record based
Mem. Mgmt.	Custom/Auto	Custom/Auto	Auto

Table: Batch vs streaming

Kafka Streams

- Stworzone w celu budowania aplikacji (mikroserwisów) pracujących bezpośrednio z danymi dostępnymi w klastrze Kafki
- Logika zdefiniowana przez topologie
- Wprowadza pojęcie partycjonowanych strumienie odpowiadający partycjon topiców
- Pojedyncza krotka w strumieniu odpowiada komunikatowi w topicu
- Partycjonowanie jest zawsze spójne wg przyjętego klucza

Kafka Streams



Kafka vs Sqoop vs Flume

Kafka

- Służy do przetwarzania srumieni real-time pomiędzy usługami
- Model Pub/sub, Dane podzielone na topiki

Sqoop

- Przerzucanie paczek danych pomiędzy Hadoop i bazami danych dla których istnieje connector (np. HBase)
- Domyślnie operuje na plikach tekstowych
- Proces zatrzymuje się po przesłaniu paczki danuych

Flume

- Zaprojektowany do zrzucania do HDFS danych takich jak logi, ruch sieciowy etc.
- Wiele do wielu konsumentów i producentów
- Przesyła dane na bieżąco w miarę ich dostępności

HBase

- Pochodzi z Bigtable (Google)
- Podstawową abstrakcją jest mapa
  $(row : string, column : string, time : int64) \longrightarrow string$
- Dane są przechowywane wg porządku klucza z możliwością partycjonowania
- Kolumny są grupowane w rodziny (*column families*)
- Przechowaywanie timestamp pozwala na wersjonowanie danych

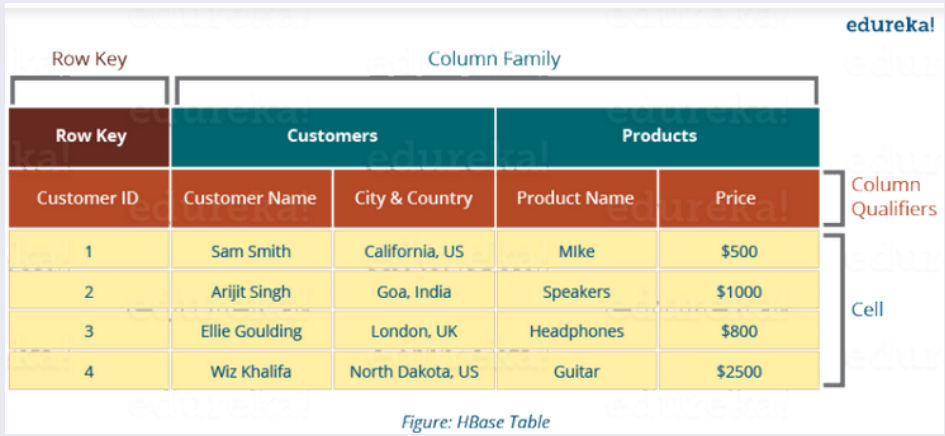


Figure: HBase Table

HBase

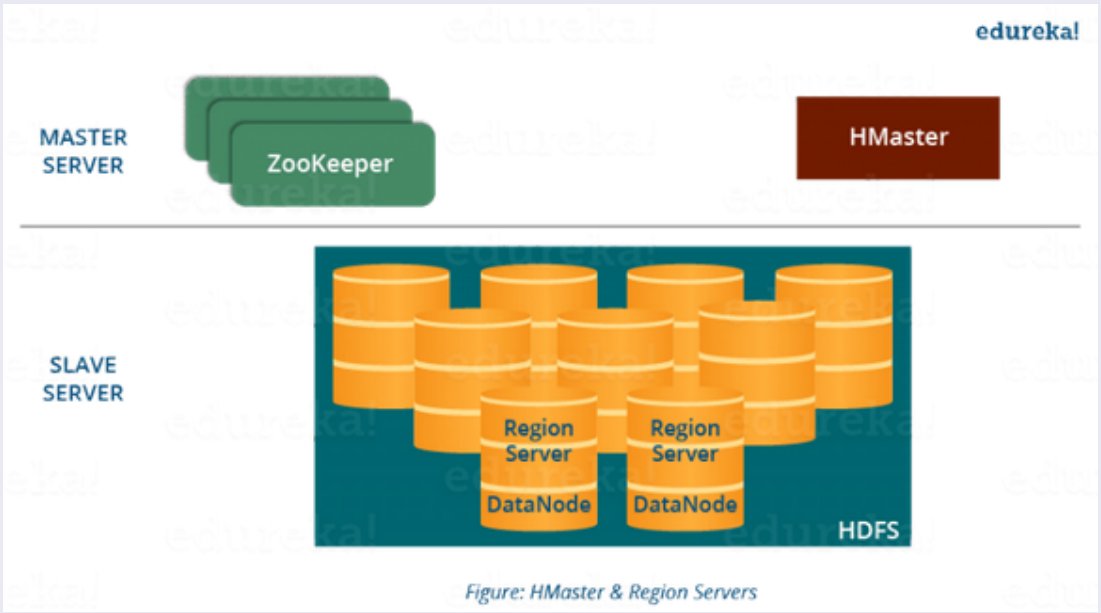
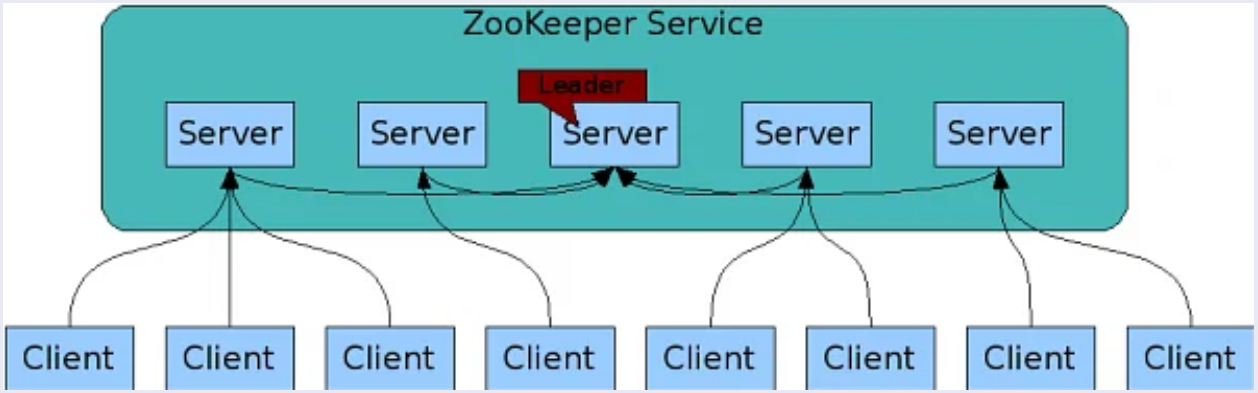


Figure: HMaster & Region Servers

- HBase Region - zawiera wszystie wiersze (posortowane) w przypisanym mu zakresie klucza
- Column family - są trzymane w jednym regionie

Zookeeper

Zookeeper jest systemem konfiguracji, synchronizacji i nazwenictwa dla aplikacji rozproszonych.



Pojęcia

- Barrier - Blokuje przetwarzanie zestawu węzłów do momentu spełnienia warunku
- Queue - Utrzymywanie kolejki
- Lock - Synchroniczne globalnie

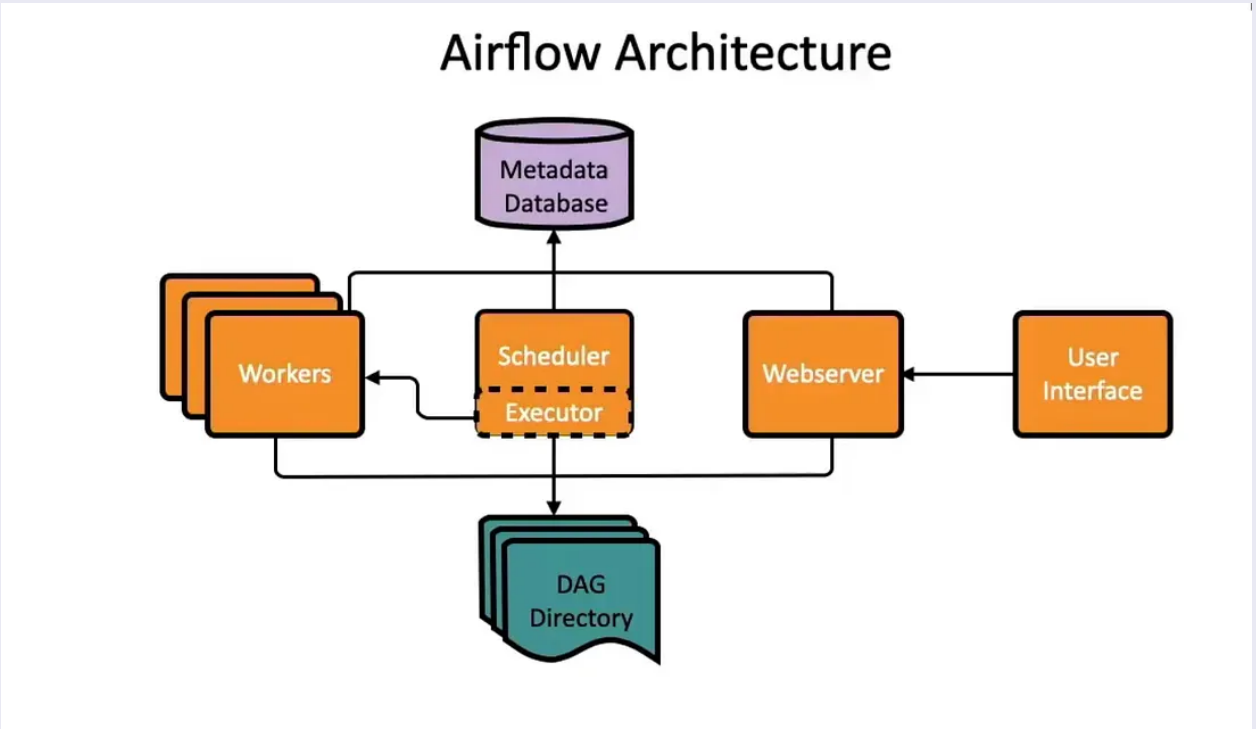
Zookeeper

Obecnie Zookeeper jest częściowo zastępowany przez Koobernetes, Raft (Kafka) etc.

Funkcja	Dawniej	Obecnie
Coordination	ZooKeeper	Raft-based internal systems
Service discovery	ZooKeeper	Kubernetes DNS / Consul
Config management	ZooKeeper	ConfigMaps / etcd / Vault
Leader election	ZooKeeper	Raft / K8s leases
Cluster metadata	ZooKeeper	Embedded metadata quorum

Airflow

Platforma open-source do planowania, uruchamiania i monitorowania złożonych workflowów danych. Pozwala definiować procesy jako kod (DAG – Directed Acyclic Graph).



Airflow

Dlaczego ważny?

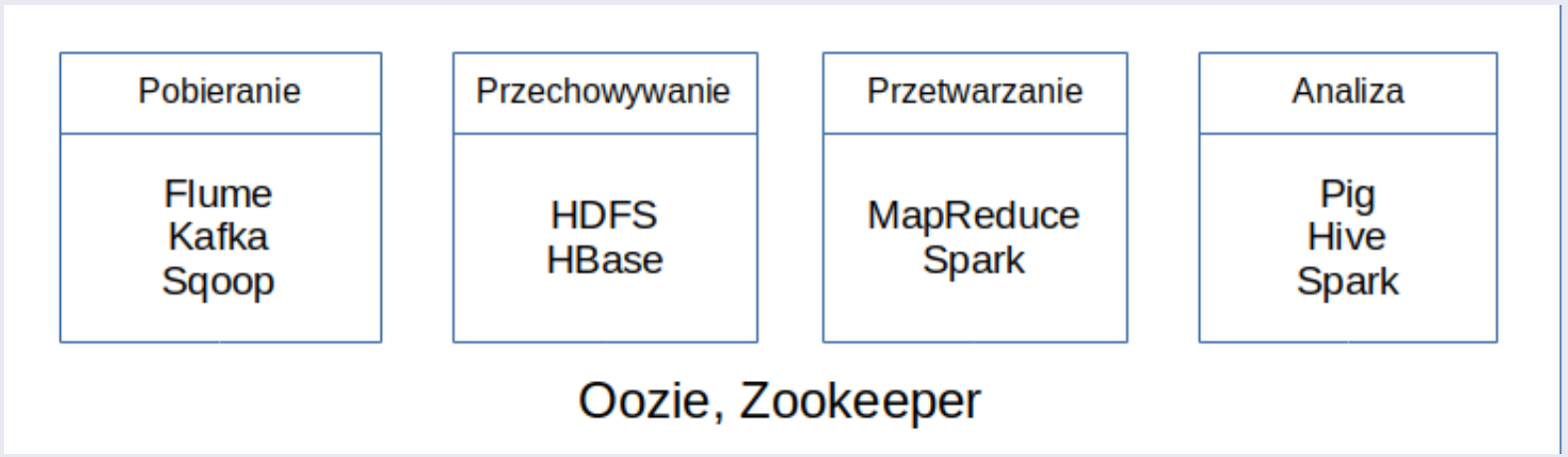
- Stał się de facto standardem w orkiestracji pipeline'ów big data i ETL/ELT.
- Oddziela logikę procesów od harmonogramowania.
- Integruje się z ekosystemem Hadoop, Spark, Hive, Presto, Snowflake, DBT, Kubernetes i wieloma innymi.

Kluczowe cechy:

- Workflow jako kod (Python) – pełna wersjonowalność.
- DAG reprezentuje zależności i kolejność zadań.
- Adaptery gotowe do użycia (Spark, Bash, Hive, Kubernetes, REST itp.).
- Airflow Scheduler zarządza kolejnością i retry.
- Web UI do monitoringu, logów i ponownego uruchamiania zadań.

Etapy przetwarzania Big Data

Etapy przetwarzania Big Data



OLAP (OnLine Analytical Processing) w świecie Big Data

OLAP (OnLine Analytical Processing)

OLAP - Analiza w czasie rzeczywistym dużych wolumentów danych przy pomocy złożonych zapytań. Cechy:

- Wielowymiarowość
- Agregacja
- Interakcja z danymi

Narzędzia OLAP







Apache Pinot

Pinot

Apache Pinot jest rozproszonym systemem OLAP działającym w czasie rzeczywistym. Pozwala na pobieranie danych ze źródeł batchowych i strumieniowych (m.in. Apache Kafka, Amazon Kinesis, Hadoop HDFS, Amazon S3, Azure ADLS, and Google Cloud Storage) oraz natychmiastowe ich odpytywanie.

Architektura Pinot

